

Xiuyuan Qiu

Kevin Tan

CSC 482

Lab 4 - IRC Chatbot

This lab focused on creating an IRC chatbot that responds to basic commands and some conversational greeting protocol.

Chatbot Description

The core of chatbot itself is built using the prebuilt example implementation provided in the lab specifications. This allows for the chatbot to connect to the chatroom and then read and write messages into that chatroom.

The main loop of the chatbot can be seen in three different states. State one, the grace period, is roughly twenty seconds long and is when the bot will only accept command input and will not enter the greeting protocol specified in phase two of the lab. State two is similar to state one, except that it will enter state three, the greeting protocol, if a “Hi” or “Hello” command is given to the bot. This is also when the timer for determining when the chatbot should reach out to any user begins. If no one reaches out to the bot in twenty seconds, it will enter state three, otherwise state two is reset every time someone asks the chatbot to do something, resetting the timer as well. If the chatbot is greeted by one of the users in the channel or the twenty second timer is reached, the chatbot will put itself into state three, and execute the greeting protocol based on which speaker it is in the conversation. After the greeting protocol is finished, the chatbot resets back to state one, allowing a grace period again to give it commands without worrying about entering the greeting protocol again right away.

Phase I: Protocols

Phase I mainly involves learning how to work with the irc spec and requires us to implement basic responses to simple user queries like “usage”, “hello”, and “users”. We needed to manage the irc protocol and send out messages in the correct format. To keep track of users, we looked at the message containing the **NAMES list** as well as the **JOIN** and **QUIT** messages.

Phase II: Greeting Protocol

Phase two, the greeting protocol, was implemented using a state machine made up of various nodes. Each node in the state machine corresponded to a different state of the greeting

protocol provided. There are two types of state nodes, **GUNode** (GiveUppableNode) and **Node**. **Nodes** store one forward connection to another node and that state's message. They correspond to the nodes with one forward connection on the state diagram. The **GUNode** is similar but also stores a node to go to if the give up timer expires, allowing for the node to switch which state to be in based on whether it got a response or timed out.

The process begins by setting the current state of the state machine to its respective start node depending on which speaker it is assigned. Every time the chatbot receives a message when in the greeting protocol, that message is sent to the state machine. If the message was sent by the user the chatbot is having a conversation with, then the current state will be set to the next state based on the current **Node**. If the message received was caused by timing out, then the current state will be set to the give up state pointed to by the **GUNode**.

Phase III: Specific QA

Kevin Tan | *League of Legends Trivia*

One extra feature of this chatbot is that it has the ability to provide trivia for League of Legends champions or provide that champion's corresponding Loldle answer. These features work by asking the chatbot about any champion in the game, and from that it will respond accordingly.

Because there are many champions and some names being harder to spell than others, a name similarity feature was implemented using the Python library, TheFuzz, to find the Levenshtein distance between the possible names and the actual names of champions. This makes using the bot a lot more flexible as you do not need exact spelling to use it. However, a problem that I did run into when implementing this was struggling with named entity recognition. I was initially using Spacy's named entity recognition model to gather the names and then do similarity testing from there, however the model was not able to identify some of the more unique names, like Mordekaiser or Heimerdinger. Because of this, rather than do NER on to find the champion names, the input message was word tokenized and a similarity score was found between all of the words. This made it so that multiple champion's names could be identified if it was at least 85% correct. If no name was provided that was at least 85% correct, then the chatbot would identify which word token was the most similar to a champion name and continue on using that as the intended champion. I chose to do this to allow flexibility in how the chatbot can be asked for the trivia. This made it feel more realistic talking to a person rather than a bot. Once the names were found, all it meant then was to retrieve the data for trivia or Loldle for that champion.

For the trivia, the Python requests library was used to get the HTML of the official League of Legends champion wiki page. Then using BeautifulSoup4, the section related to the champion's trivia was gathered and parsed. After gathering all of the searched champion's trivia points, a random sentence was chosen to be returned to the user. For the Lolde answer, the champion is searched up from a CSV file of all champions and their respective attributes. All of the data that was in the CSV was found by scraping the Loldle website and seeing how they categorized each champion. Several sentences are then formed based on the aspects of those attributes, describing the champions according to what is needed for Loldle.

Xiuyuan Qiu | *Cal Poly CSC Office Hours, Locations, and Emails*

The extra feature I implemented is a query of the current office hours for the CSC department. There is a webpage on the CSC department website (<https://csc.calpoly.edu/faculty/>) that has a table containing the Fall office hours of CSC department professors.

To scrape the webpage, I use Python's urllib.request library to grab the HTML from the webpage, before using BeautifulSoup4 to select all tables (selector `div > table`), and find the table that has 'Office Hours' in the key (first row). That table is then converted into a list of lists by my function `table2arr()` to be used to respond to user requests.

The query that the feature expects can be expressed with the following BNF (where `<>` is a variable and `()` is optional):

```
Query ::= (<Q>) (<IS>) (<TITLE>) <PROF>('s) <PROP>(?)

Q ::= what
    | where
    | when

IS ::= is
    | are

PROP ::= oh
        | office hours
        | hours
        | office
        | room
        | room number
        | email

PROF ::= <first name>
        | <last name>
        | <full name>

TITLE ::= Dr(.)
        | Prof(.)
```

For example: “what is dr Khosmood’s office” or “beard oh” or “Ayaan’s email?”. Note that casing does not matter in the query. In the irc channel, the queries must be prefixed by `<bot’s name>: !oh`.

The strategy for parsing user queries is to reduce the query to its simplest form, stripping away all optional values and only looking at `<PROF>` and `<PROP>`. We match the tail of the query to `<PROP>` values; if there is no match, the request is invalid and we let the user know. If it is valid, we take the rest of the reduced string as the professor name and look in our office hours table for a professor that has a first, last, or full name that matches that string. If there are multiple matches for first or last name, we produce an error that asks the user to be more specific in the query and show some of the matching names. If there is one match, or the full name matches, we return the requested `<PROP>`. If no matches occur, we ask the user to check their spelling and give a link to the source webpage in case they don’t know the correct spelling.

There is one edge case to handle: “Where is ____’s office hours” has `<PROP>` ‘office hours’, but is asking for room number and not hours. So if `<Q>` is “where”, and `<PROP>` refers to an office hour, we will change the type of the query to ‘room’ instead of ‘office hours’. This is the only edge case, as other similar cases (when is __’s email, when is __’s room) are not well formulated, so our response can use the `<PROP>` to decide what to return.